

Naive Bayes Classifier in R: Bayes' Theorem and Probability Explained

Bayesian Classification

Among the probabilistic model families, Bayesian classifiers stand out by making probabilistic predictions — estimating the likelihood that an observation belongs to each class. This powerful method combines prior knowledge with observed data to make well-informed decisions.

Theoretical Foundation

Grounded in Bayes' Theorem, one of the cornerstones of modern statistical inference.

Performance

The naïve Bayesian classifier delivers performance comparable to selected decision trees and neural networks.

Incremental Learning

Each training example can incrementally increase or decrease the probability that a given hypothesis is correct.

Even when Bayesian methods are computationally intractable, they can serve as an optimal decision-making benchmark against which other methods can be measured.

Bayes' Theorem: The Mathematical Foundation

At the heart of Bayesian classification lies Bayes' Theorem — the mathematical framework that makes probabilistic inference possible. It allows us to continuously update our beliefs about the likelihood of an event as new evidence emerges.

Essential Components

- **P(H)**: Prior probability of hypothesis H
- **P(X)**: Prior probability of evidence X
- **P(H|X)**: Posterior probability of H given X
- **P(X|H)**: Probability of X given H (likelihood)

The elegance of this theorem lies in its ability to invert conditional probabilities — enabling sophisticated probabilistic reasoning from the ground up.

Bayes' Theorem:
$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

Putting Bayes' Theorem to Work: Prediction

Building on this mathematical foundation, Bayesian classification applies the theorem to compute the probability that a given data point belongs to each possible class — turning abstract inference into actionable predictions.

1

Define the Prior

Establish initial probabilities $P(c_i)$ for each class $c_i \in C$ using the training data as a starting belief.

2

Compute the Likelihood

Determine $P(X|c_i)$ — the probability of observing features X given that the class is c_i .

3

Apply Bayes' Theorem

Use the theorem to derive $P(c_i|X)$ — the posterior probability of class c_i given the observed features X .

4

Classify

Assign the instance to the class with the highest posterior probability — the model's most informed decision.

This probabilistic framework allows the classifier to go beyond a simple label: it quantifies uncertainty at every step, delivering not just a predicted class, but a calibrated measure of confidence in that prediction.

Classification via Maximum A Posteriori

Building on that posterior probability, the goal of Bayesian classification is to derive the **Maximum A Posteriori (MAP)** estimate — the most likely hypothesis given the observed data.

MAP Formulation

Select the class c_i that maximizes $P(c_i|X)$, where X represents the observed features.

Since $P(X)$ is constant across all classes, we can simplify the comparison by evaluating only $P(X|c_i) \times P(c_i)$.

📌 **Key Simplification:** In practice, $P(X)$ can be safely ignored during classification, as it is identical for every class. We only need to compare the numerators: $P(X|c_i) \times P(c_i)$.

The Naïve Bayes Classifier

Naïve Bayes is a probabilistic classifier grounded in Bayes' theorem. It assigns a new instance to the class that maximizes the posterior probability $P(c_i|X)$. Its defining assumption — **conditional independence of features given the class** — makes the model computationally tractable even in high-dimensional spaces.

The Conditional Independence Assumption

Given a class c_i , the joint likelihood of observing features $x_1, x_2, \dots, x_n$ is decomposed as a product of individual likelihoods:

$$P(X|c_i) = \prod_{k=1}^n P(x_k|c_i)$$

This factorization reduces the estimation problem from modeling a full joint distribution over all features to estimating n independent univariate distributions — one per feature per class.

Classification Procedure

01

Compute Prior Probabilities

Estimate $P(c_i)$ as the relative frequency of class c_i in the training set.

03

Compute Posterior Scores

For a new instance X , compute $P(c_i) \prod_k P(x_k|c_i)$ for every class c_i .

02

Estimate Conditional Probabilities

For each feature x_k and each class c_i , estimate $P(x_k|c_i)$ from training data.

04

Assign the Winning Class

Classify X into the class $c^* = \arg \max_{c_i} P(c_i) \prod_k P(x_k|c_i)$.

Key Properties



Small Datasets

Performs well even with limited training data, requiring few examples per class to estimate reliable probabilities.



Mixed Feature Types

Handles both categorical features (via frequency counts) and continuous features (via Gaussian or kernel density estimation).



Robust to Noise

Irrelevant features have limited impact on classification, as each feature contributes independently and proportionally.



Fast & Scalable

Training and prediction are both linear in the number of features and training instances, enabling real-time classification.

Training Dataset

To ground our classification concepts in practice, consider this real-world training dataset. The goal is to predict whether a customer **Buys Computer** — our target class — based on four key attributes: **Age**, **Income**, **Student**, and **Credit Rating**. With 14 labeled examples, this compact dataset captures a meaningful variety of customer profiles and purchasing behaviors.

Dataset at a Glance

- 14 training instances
- 4 predictor attributes
- 1 binary target variable
- Mix of categorical attributes

Class Distribution

- **Buys Computer = Yes:** 9 instances (64%)
- **Buys Computer = No:** 5 instances (36%)

Reference: J. Han, J. Pei, and H. Tong, Data Mining: Concepts and Techniques, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.

Age	Income	Student	Credit Rating	Buys Computer
≤30	high	no	fair	no
≤30	high	no	excellent	no
31-40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31-40	low	yes	excellent	yes
≤30	medium	no	fair	no
≤30	low	yes	fair	yes
>40	medium	yes	fair	yes
≤30	medium	yes	excellent	yes
31-40	medium	no	excellent	yes
31-40	high	yes	fair	yes
>40	medium	no	excellent	no

Naïve Bayes Classification: A Worked Example

Now that we understand the algorithm's steps, let's see it in action by classifying a new instance: **X = (age = ≤30, income = medium, student = yes, credit_rating = fair)**

1

Step 1: Compute Prior Probabilities $P(\text{buys_computer})$

$$P(\text{yes}) = 9/14 = 0.643$$

$$P(\text{no}) = 5/14 = 0.357$$

2

Step 2: Compute the Likelihood $P(X|\text{yes})$

$$\begin{aligned} &P(\text{age} \leq 30 | \text{yes}) \times P(\text{income} = \text{medium} | \text{yes}) \times P(\text{student} = \text{yes} | \text{yes}) \times P(\text{credit_rating} = \text{fair} | \text{yes}) \\ &= (2/9) \times (4/9) \times (6/9) \times (6/9) = 0.044 \end{aligned}$$

3

Step 3: Compute the Likelihood $P(X|\text{no})$

$$\begin{aligned} &P(\text{age} \leq 30 | \text{no}) \times P(\text{income} = \text{medium} | \text{no}) \times P(\text{student} = \text{yes} | \text{no}) \times P(\text{credit_rating} = \text{fair} | \text{no}) \\ &= (3/5) \times (2/5) \times (1/5) \times (2/5) = 0.019 \end{aligned}$$

4

Step 4: Compare the Posterior Probabilities

$$P(X|\text{yes}) \times P(\text{yes}) = 0.044 \times 0.643 = 0.028$$

$$P(X|\text{no}) \times P(\text{no}) = 0.019 \times 0.357 = 0.007$$



Result: Since $P(X|\text{yes}) \times P(\text{yes}) > P(X|\text{no}) \times P(\text{no})$, we classify X as $\text{buys_computer} = \text{YES}$

Naïve Bayes in Practice: `cla_nb` (daltoolbox)

The `daltoolbox` R package offers a unified and consistent interface for various classification models, simplifying their application in data analysis workflows. Within this comprehensive framework, the `cla_nb` function specifically implements the Naïve Bayes classifier, allowing users to leverage its probabilistic approach with ease.

Function Signature

`cla_nb(attribute, slevels)`

- `attribute`: A string specifying the name of the target column (the class variable).
- `slevels`: A vector containing the labels of the class variable.

Workflow

The `cla_nb` function integrates seamlessly into the standard `daltoolbox` machine learning pipeline:

1. **Instantiate:** Create an instance of the Naïve Bayes model.
2. **Fit:** Train the model using the `fit()` method on your training data.
3. **Predict:** Generate predictions on new data using the `predict()` method.
4. **Evaluate:** Assess model performance using the `evaluate()` method.

Under the Hood

The `cla_nb` implementation:

- Utilizes **Gaussian Naïve Bayes** for handling continuous features, assuming a normal distribution within each class.
- Estimates the **prior probabilities** $P(\text{class})$ and **conditional probabilities** $P(\text{feature}|\text{class})$ directly from the provided training data.

📌 **Practical Note:** No mandatory hyperparameters are required. The model is ready to use out of the box, making it ideal for quick baselines and benchmarking.

Naive Bayes: cla_nb

```
# Carregando o pacote
library(daltoolbox)

# Dados e níveis
iris <- datasets::iris
slevels <- levels(iris$Species)

# Divisão treino/teste
set.seed(1)
sr <- sample_random()
sr <- train_test(sr, iris)
iris_train <- sr$train
iris_test <- sr$test
```

```
# Treinamento
model <- cla_nb("Species", slevels)
model <- fit(model, iris_train)

# Avaliação — treino
train_pred <- predict(model, iris_train)
train_eval <- evaluate(model,
  adjust_class_label(iris_train[, "Species"]),
  train_pred)
# acurácia: 0.958

# Avaliação — teste
test_pred <- predict(model, iris_test)
test_eval <- evaluate(model,
  adjust_class_label(iris_test[, "Species"]),
  test_pred)
# acurácia: 0.967
```

Step-by-Step Walkthrough

- Instantiate with `cla_nb("Species", slevels)` — no mandatory hyperparameters
- Probabilistic model: estimates $P(\text{class} \mid \text{features})$ via Bayes' theorem
- Assumes conditional independence between features — the "naive" premise
- Train accuracy: **0.958** | Test accuracy: **0.967** — slightly higher on the test set, common with small datasets
- Simple, efficient, and surprisingly competitive across many tasks

📌 **Conceito-chave:** Naive Bayes é um **modelo probabilístico** que aplica o teorema de Bayes assumindo independência entre as features. Permanece surpreendentemente competitivo mesmo quando essa suposição é violada na prática.

A Second Probabilistic Family Member – `cla_multinom`

Multinomial logistic regression for multiclass classification — a probabilistic linear model that naturally handles more than two classes, unlike binary logistic regression.

```
model <- cla_multinom("Species")
model <- fit(model, iris_train)
prediction <- predict(model, iris_test)
evaluate(model, iris_test$Species,
         prediction)
```

- **Didactic goal:** show a probabilistic linear model that handles more than two classes directly, without one-vs-rest workarounds.
- **Same rhythm:** configure → fit → predict → evaluate, exactly like `cla_nb`.
- **Contrast with Naive Bayes:** both are probabilistic, but Naive Bayes assumes conditional independence between features, while multinomial regression models class log-odds as a direct linear combination of the features.

Reference: Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

